

DEMO · PART A COMPANION

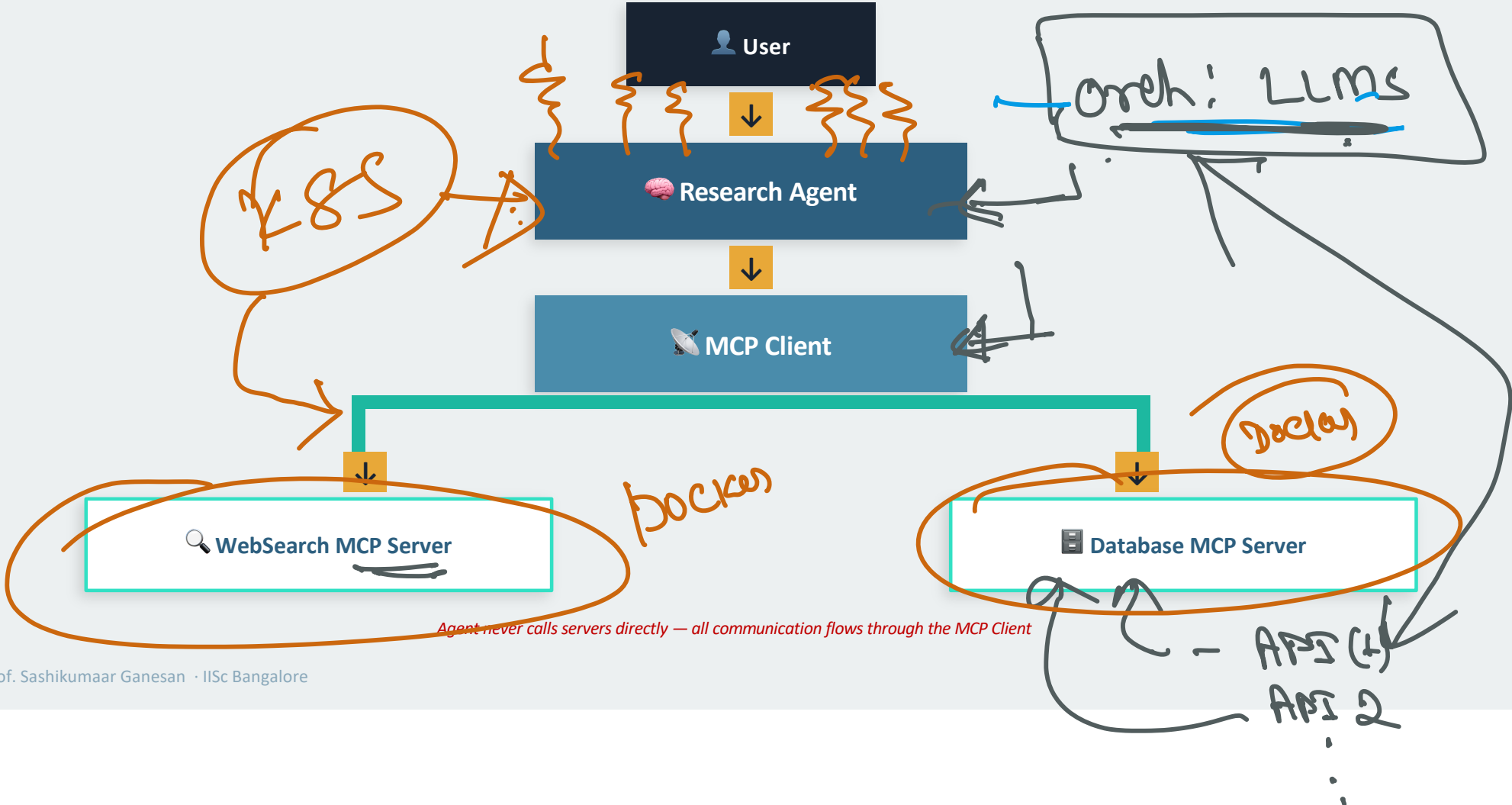
MCP Agent Demonstration

1 Agent · 1 MCP Client · 2 MCP
Servers

Research Agent · WebSearch Server · Database Server

The Minimal MCP Platform: What We're Building

Smallest real mental model — 1 Agent · 1 Client · 2 Servers



Step 1: Define the MCP Servers

Each server is an isolated service that exposes tools — the agent never calls these directly

websearch_server.py

```
class WebSearchServer:
```

```
    def search_news(self, query):
        # Returns list of news headlines
        return [
            f"News result 1 for {query}",
            f"News result 2 for {query}"
        ]
```

```
# Exposes ONE tool: search_news(query)
# In production: runs in its own Docker
# container, reachable via HTTP/SSE
```

database_server.py

```
class DatabaseServer:
```

```
    def get_company_info(self, company):
        # Returns structured company record
        return {
            "company": company,
            "sector": "Automotive",
            "country": "USA"
        }
```

```
# Exposes ONE tool: get_company_info()
# In production: PostgreSQL / internal DB
```

Key insight: each server knows nothing about the agent — it just exposes tools and waits to be called

Step 2: The MCP Client — Communication Layer

The client is the universal router — agents call it, it finds the right server and tool

mcp_client.py



```
class MCPClient:

    def __init__(self):
        self.servers = {}

    def register_server(self, name, server):
        # Add a server to the registry
        self.servers[name] = server

    def call_tool(self, server_name,
                  tool_name, **kwargs):
        server = self.servers[server_name]
        tool = getattr(server, tool_name)
        return tool(**kwargs) # execute!
```

register_server()

①

Plugs a server into the client registry. In production this is service discovery over the network.

call_tool()

②

3-step dispatch: look up server → find the method by name → call it with keyword arguments.

Why this matters

③

The agent calls ONE interface. The client handles routing, retries, and protocol — agent code never changes when you swap servers.

Step 3: The Research Agent — Decision Maker

The agent decides what to do. It never calls servers — it calls the MCP client

research_agent.py

```
class ResearchAgent:

    def __init__(self, client):
        self.client = client # injected

    def analyze_company(self, company):
        # Tool Call 1 - company facts
        company_info = self.client.call_tool(
            server_name = "database",
            tool_name = "get_company_info",
            company = company
        )
        # Tool Call 2 - latest news
        news = self.client.call_tool(
            server_name = "websearch",
            tool_name = "search_news",
            query = f"{company} earnings"
        )
        return {company_info, news}
```

A

Dependency Injection

Client is passed in at construction — the agent has zero knowledge of server addresses, protocols, or credentials.

B

Tool Call 1: database

Agent requests structured company data. Specifies server, tool, and keyword argument — nothing else.

C

Tool Call 2: websearch

Agent composes a news query and routes it to the websearch server — same interface as the database call.

Notice: agent code is identical whether servers run locally, in Docker, or in the cloud — the MCP Client abstracts all of that

Step 4: Wiring Everything Together

Four lines to bootstrap the entire platform — create, register, connect, run

main.py — bootstrap

```
from websearch_server import WebSearchServer
from database_server import DatabaseServer
from mcp_client import MCPClient
from research_agent import ResearchAgent

# — 1. Create the servers —————
web_server = WebSearchServer()
db_server = DatabaseServer()

# — 2. Create the client —————
client = MCPClient()

# — 3. Register servers —————
client.register_server("websearch", web_server)
client.register_server("database", db_server)

# — 4. Create + run agent —————
agent = ResearchAgent(client)
result = agent.analyze_company("Tesla")
print(result)
```

1

Create servers

Instantiate each MCP server object. In prod: docker run each server independently.

2

Create client

One MCPClient instance per agent session. Starts with an empty server registry.

3

Register servers

Two register_server() calls plug both services in. In prod: service discovery auto-populates this.

4

Run agent

Agent receives the wired client and immediately has access to both tools — no other setup needed.

The wiring code is the only place that knows about all four components — agents, servers, and client stay decoupled

Agent 1

Agent 2

Client A

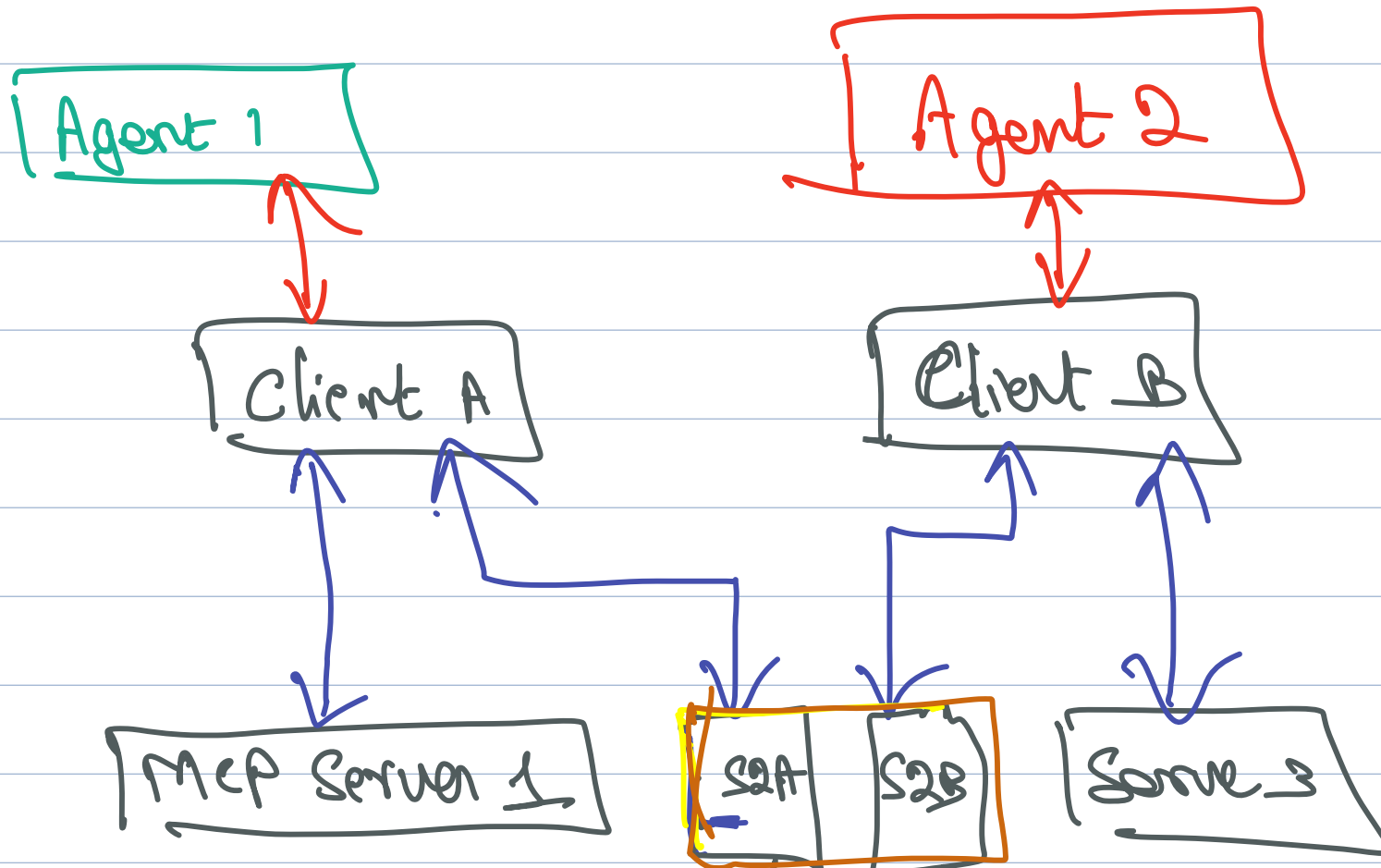
Client B

MC Server 1

S2A S2B

Server 3

instant of the same class but diff object(s).



What Happens Internally: Step-by-Step Trace

User input: "Analyze Tesla"



Each `call_tool()` is a discrete round-trip: route → execute → return · No shared state between tool calls

OUTPUT

The Result: Structured Agent Output

Two tool calls, two servers, one unified result — assembled by the agent

print(result) → agent output

```
{
  "company_info": {
    "company": "Tesla",
    "sector": "Automotive",
    "country": "USA"
  },
  "latest_news": [
    "News result 1 for Tesla earnings",
    "News result 2 for Tesla earnings"
  ]
}
```

Where Did Each Field Come From?

company_info{ }

Server: DatabaseServer

Tool: get_company_info()

latest_news[]

Server: WebSearchServer

Tool: search_news()

In production: the agent's LLM interprets these results and generates a natural-language analysis report — same two tool calls, richer synthesis.

The agent assembled this from two independent tool calls — neither server knew about the other

Simulated Code → Real MCP: What Changes?

The logical roles stay identical — only the transport layer changes

Simulated (this demo)

```
agent
  ↓ call_tool()
MCPClient
  ↓ getattr(server, tool)()
WebSearchServer (Python object)
  ↓ return list

DatabaseServer (Python object)
  ↓ return dict
```

Real MCP (production)

```
agent
  ↓ call_tool()
MCPClient
  ↓ JSON-RPC 2.0 / HTTP-SSE
WebSearch MCP Server (Docker)
  ↓ JSON response over network

Database MCP Server (Docker)
  ↓ JSON response over network
```

Aspect	This Demo	Real MCP (FastMCP / Claude)
Transport	Direct Python method call	JSON-RPC 2.0 over HTTP-SSE or stdio
Server location	Same process	Separate Docker containers
Tool discovery	Hardcoded names	Server advertises schema at startup
Auth	None	OAuth 2.0 bearer tokens
Agent code	Identical ✓	Identical ✓

Inside call_tool(): What the Client Actually Sends

Your simplified call_tool() hides a full JSON-RPC 2.0 request/response exchange

What the Agent writes

```
company_info = client.call_tool(
    "database",
    "get_company_info",
    company = "Tesla"
)
```

hides
↓

What the Protocol actually sends

```
{
  "jsonrpc": "2.0",
  "id": 101,
  "method": "tools/call",
  "params": {
    "name": "get_company_info",
    "arguments": { "company": "Tesla" }
  }
}
```

Server response travels back through the protocol

```
{ "jsonrpc": "2.0", "id": 101, "result": { "company": "Tesla", "sector": "Automotive", "country": "USA" } }
```

id: 101 matches request to response — the client uses this to correlate async replies

tools/list — How Agents Discover Available Tools

Before ordering, the client reads the menu — tools/list is that menu request

Client sends: tools/list request

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list"
}
```

Server responds: available tools schema

```
{ "result": { "tools": [
  { "name": "get_company_info",
    "description": "Fetch company record",
    "inputSchema": { "company": "string" } },
  { "name": "get_financials" }
] } }
```

The tool lifecycle: discover → choose → call → receive

1 tools/list

Client asks server what tools exist and their input schemas

2 Agent decides

LLM reads schema, selects the right tool and fills the arguments

3 tools/call

Client sends the JSON-RPC request with the chosen tool + arguments

4 result

Server executes, wraps output in a JSON-RPC response, client unwraps it

In FastMCP and Claude Desktop: tool discovery happens automatically at session start — no manual call needed

The 5 MCP Method Primitives — One Analogy Each

Every interaction between an MCP client and server uses one of these five methods



tools/list

Open the menu

Ask the server what executable functions it exposes

At session start — discover what actions are available

```
{ "method": "tools/list" } → [ "get_company_info", "search_news", ... ]
```



tools/call

Order an item

Invoke a specific tool with arguments — causes a side-effect or returns data

When the agent decides it needs an action performed

```
{ "method": "tools/call", "params": { "name": "search_news", "arguments": { "query": "Tesla" } } }
```



resources/list

Browse the library catalog

Ask the server what read-only data sources (URIs) are available

When the agent needs to know what documents/data it can access

```
{ "method": "resources/list" } → [ "file:///reports/q2.md", "db://schema" ]
```



resources/read

Read a book

Fetch the contents of a specific resource URI — read-only, no side effects

When the agent needs to inject a document into its context

```
{ "method": "resources/read", "params": { "uri": "file:///reports/q2.md" } }
```



prompts/list

View available templates

Discover reusable prompt templates the server exposes for common workflows

```
{ "method": "prompts/list" } → [ "summarize_ticket", "draft_reply", "explain_code" ]
```

tools = actions with side effects · resources = read-only data · prompts = reusable templates · all use JSON-RPC 2.0

TWO CALLS

Both Protocol Calls Side-by-Side: DB + WebSearch

Same JSON-RPC structure — different id, different server, different tool

Call 1 → Database MCP Server

```
{ Request }
  "id": 101,
  "method": "tools/call",
  "params": { "name": "get_company_info",
    "arguments": { "company": "Tesla" } }
```

```
{ Response }
  "id": 101,
  "result": { "company": "Tesla",
    "sector": "Automotive", "country": "USA" }
```

Call 2 → WebSearch MCP Server

```
{ Request }
  "id": 102,
  "method": "tools/call",
  "params": { "name": "search_news",
    "arguments": { "query": "Tesla earnings" } }
```

```
{ Response }
  "id": 102,
  "result": { "articles": [
    "Tesla beats estimates", "Analysts upgrade" ] }
```

id: 101 → db server · id: 102 → websearch server · each round-trip is independent and correlation-tracked by id

Structure is identical across ALL MCP tool calls — **jsonrpc + id + method + params** — this is why any client can talk to any server

Complete Protocol Sequence: User to Final Answer

Every arrow is either a Python call (above protocol) or a JSON-RPC message (inside protocol)



The Protocol Layer: What It Is and Why It Matters

The standardised JSON messages and rules that let any MCP client talk to any MCP server



tools/list

Client asks the server what tools it exposes. Server returns name + description + inputSchema for each tool.

Analogy:

Opening the restaurant menu



tools/call

Client invokes a specific tool with arguments. The core message of the protocol — this is what makes things happen.

Analogy:

Placing your order



result / error

Server wraps execution output in a JSON-RPC result, or an error object if something went wrong. id field links back to the request.

Analogy:

Food arrives at the table (or kitchen calls back)

The protocol layer = **standardised JSON-RPC messages + communication rules** that allow any MCP client to talk to any MCP server, regardless of language, framework, or location.

The Minimal MCP Mental Model



Agent:

Decides which tools to call and in what order. Knows nothing about servers, protocols, or transport.



MCP Client:

Universal router. Translates `call_tool()` → JSON-RPC request → waits for response → hands Python object back to agent.



Protocol Layer:

Standardised JSON-RPC 2.0 messages: `tools/list` (discover) · `tools/call` (invoke) · `result` (respond). Any client, any server.



MCP Servers:

Isolated services. Receive JSON-RPC, execute tools, return structured result. Run in separate containers in production.

Agent decides → Client encodes JSON-RPC → Protocol transports → Server executes → Result returns